CSE563: Unit VI Study Notes

Algorithmic and Optimization Approaches

School of Computer Science & Engineering

Session 2025–26

Contents

1 Unit VI: Algorithmic and Optimization Approaches **2**
- 1.1 Greedy Approach . . . 2
- 1.2 Divide and Conquer . . . 3
- 1.3 Dynamic Programming . . . 4
- 1.4 Backtracking . . . 6
- 1.5 Genetic Algorithms . . . 7
- 1.6 Nature-Inspired Algorithms and Approaches . . . 8
- 1.7 Unit VI Quick-Revision Sheet . . . 10

1 Unit VI: Algorithmic and Optimization Approaches

Course Outcome CO6: discuss different optimization approaches to solve mathematical problems.

Unit V told us how expensive a solution is; this unit is about *how to design one*. Four classical paradigms (greedy, divide and conquer, dynamic programming, backtracking) give exact answers with different cost profiles. When the problem is NP-hard and large, two families of *metaheuristics* (genetic algorithms and other nature-inspired methods) trade the guarantee of optimality for good solutions in reasonable time.

Choosing a Paradigm

Paradigm	Idea	Optimal?	Use when
Greedy	Best local choice, never revisit	If greedy-choice property holds	Provably safe choices (MST, Huffman, activity selection)
Divide & conquer	Split, solve independently, combine	Yes	Subproblems independent (sorting, multiplication)
Dynamic programming	Solve overlapping subproblems once	Yes	Optimal substructure + overlapping subproblems (knapsack, LCS, shortest paths)
Backtracking	DFS over partial solutions, prune	Yes (exhaustive)	Constraint satisfaction, small NP-hard instances
Genetic / nature-inspired	Population evolved stochastically	No	Large NP-hard or black-box problems; "good enough, fast"

1.1 Greedy Approach

Definition 1.1. A **greedy algorithm** builds a solution step by step, each time taking the choice that looks best *right now*, and never reconsiders. It is correct when the problem has the **greedy-choice property** (a locally optimal choice is part of some globally optimal solution) and **optimal substructure** (an optimal solution contains optimal solutions to subproblems).

Worked Example: Activity Selection

Activities with (start, finish): a_1=(1, 4), a_2=(3, 5), a_3=(0, 6), a_4=(5, 7), a_5=(3, 9), a_6=(5, 9), a_7=(6, 10), a_8=(8, 11), a_9=(8, 12), a_{10}=(2, 14), a_{11}=(12, 16). Select the maximum number of non-overlapping activities.

Greedy rule: sort by finish time; always pick the activity that *finishes earliest* among those compatible with the last pick (it leaves the most room).

Pick a_1 (finish 4). Next compatible (start ≥ 4): a_4 (5,7). Next start ≥ 7: a_8 (8,11). Next start ≥ 11: a_{11} (12,16). **Answer: 4 activities** $\{a_1, a_4, a_8, a_{11}\}$. Time $O(n \log n)$ for the sort.

Why it works: if an optimal solution does not start with the earliest-finishing activity, swap that in for its first activity—nothing else is affected (exchange argument). Choosing by *shortest duration* or *earliest start* instead can fail: $a_3 = (0, 6)$ starts earliest but blocks a_1 and a_4.

Worked Example: Fractional Knapsack

Capacity $W = 7$. Items (weight, value): $I_1 = (1, 1)$, $I_2 = (3, 4)$, $I_3 = (4, 5)$, $I_4 = (5, 7)$. Value/weight ratios: 1.00, 1.33, 1.25, 1.40. Greedy by ratio: take all of I_4 (weight 5, value 7); remaining capacity 2; take 2/3 of I_2 (weight 2, value 2.67). **Total** = 9.67, optimal for the fractional problem.

The same greedy on 0/1 knapsack fails: whole items only: I_4 (5,7), then I_2 does not fit, I_3 does not fit, I_1 fits: total 8. But $I_2 + I_3$ gives weight 7, value **9**. 0/1 knapsack needs dynamic programming (below).

Worked Example: Huffman Coding

Symbol frequencies (per 100): a:45, b:13, c:12, d:16, e:9, f:5. Build the code by repeatedly merging the two least-frequent nodes:

1. Merge $f(5) + e(9) \to 14$.
2. Merge $c(12) + b(13) \to 25$.
3. Merge $14 + d(16) \to 30$.
4. Merge $25 + 30 \to 55$.
5. Merge $a(45) + 55 \to 100$ (root).

Label left branches 0, right branches 1: $a = 0$, $c = 100$, $b = 101$, $f = 1100$, $e = 1101$, $d = 111$. Cost $= 45(1) + 12(3) + 13(3) + 5(4) + 9(4) + 16(3) = 224$ bits per 100 symbols, versus 300 for a fixed 3-bit code: 25% saving. Huffman is optimal among prefix codes; runs in $O(n \log n)$ with a min-heap. It is the entropy-coding stage of JPEG, MP3, ZIP.

Common Pitfall & Misconception: Coin Change: Greedy Is Not Always Right

With coins $\{1, 5, 10, 25\}$ (canonical systems) greedy "largest coin first" is optimal: $63 = 25 + 25 + 10 + 1 + 1 + 1$ (6 coins). With coins $\{1, 3, 4\}$ and amount 6: greedy gives $4 + 1 + 1$ (3 coins), optimal is $3 + 3$ (2 coins). Always ask: *does the greedy-choice property hold?* If you cannot prove it, use DP.

Exam Tip & Key takeaway: Greedy Algorithms You Already Know

Dijkstra (closest unvisited vertex), Prim and Kruskal (lightest safe edge), Huffman, activity selection, fractional knapsack, Welsh–Powell colouring (heuristic, not optimal). The proof pattern is always the **exchange argument**: take any optimal solution, swap in the greedy choice, show it is no worse.

1.2 Divide and Conquer

Template

1. **Divide** the problem into a subproblems of size n/b.
2. **Conquer** each recursively (base case: solve directly).
3. **Combine** the sub-solutions in $f(n)$ time.

Running time $T(n) = aT(n/b) + f(n)$—the master theorem (Unit V) reads off the answer. The subproblems must be *independent*; if they overlap, use dynamic programming.

Worked Example: Karatsuba Multiplication

Multiply $x = 12$ and $y = 34$ (two "digits" each, base 10). Write $x = 1 \cdot 10 + 2$, $y = 3 \cdot 10 + 4$, so $a = 1, b = 2, c = 3, d = 4$. Schoolbook: $xy = ac \cdot 10^2 + (ad + bc) \cdot 10 + bd$ needs **four** products.

Problem	Divide / combine	Recurrence	Time
Binary search	One half; no combine	$T(n/2)+1$	$\Theta(\log n)$
Merge sort	Two halves; linear merge	$2T(n/2)+n$	$\Theta(n\log n)$
Quicksort	Partition, then two sides	$2T(n/2)+n$ (avg)	$\Theta(n\log n)$ avg
Max subarray	Halves + best crossing sum	$2T(n/2)+n$	$\Theta(n\log n)$
Karatsuba multiply	3 half-size products	$3T(n/2)+n$	$\Theta(n^{1.585})$
Strassen matrix multiply	7 half-size products	$7T(n/2)+n^2$	$\Theta(n^{2.807})$
Closest pair of points	Halves + strip check	$2T(n/2)+n$	$\Theta(n\log n)$
Fast Fourier Transform	Even/odd indices	$2T(n/2)+n$	$\Theta(n\log n)$

Table 1: Divide-and-conquer algorithms. Naive multiplication is $\Theta(n^2)$; naive matrix multiplication $\Theta(n^3)$.

Karatsuba: compute only **three**:

$$P_1 = ac = 3, \qquad P_2 = bd = 8, \qquad P_3 = (a+b)(c+d) = 3\times 7 = 21,$$

and recover the middle term as $ad+bc = P_3 - P_1 - P_2 = 21-3-8 = 10$. Result: $3\cdot 100 + 10\cdot 10 + 8 = 408$. ✓Replacing 4 recursive products by 3 changes $4T(n/2)+n = \Theta(n^2)$ into $3T(n/2)+n = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$. For 1000-digit numbers that is roughly 10^6 vs $\sim 5\times 10^4$ digit-multiplications.

Worked Example: Maximum Subarray by Divide and Conquer

$A = [-2,1,-3,4,-1,2,1,-5,4]$ (indices 0..8). Split at mid = 4: left = $A[0..4] = [-2,1,-3,4,-1]$, right = $A[5..8] = [2,1,-5,4]$. The answer is the best of three: best subarray entirely in the left, entirely in the right, or *crossing* the midpoint.
Crossing: best suffix of the left ending at index 4—running sums leftward from $A[4]$: $-1, 3, 0, 1, -1$, max = 3 ($[4,-1]$). Best prefix of the right starting at index 5: $2, 3, -2, 2$, max = 3 ($[2,1]$). Crossing sum = $3+3=6$.
Recursively: best purely-left = 4 ($[4]$); best purely-right = 4 ($[4]$). Maximum = **6**, subarray $[4,-1,2,1]$. Kadane's $O(n)$ DP gives the same answer in one pass, but the divide-and-conquer version illustrates the "combine" step costing $\Theta(n)$.

1.3 Dynamic Programming

When and How

Use DP when a problem has **optimal substructure** (an optimal solution is built from optimal solutions of subproblems) *and* **overlapping subproblems** (the same subproblems recur, so naive recursion re-solves them exponentially many times).

1. Define the **state**: what does dp[i] (or dp[i][j]) mean?
2. Write the **recurrence** relating a state to smaller states.
3. Set the **base cases**.
4. Choose an **evaluation order**: *memoisation* (top-down recursion + cache) or *tabulation* (bottom-up loop).
5. **Reconstruct** the solution by tracing back which choice produced each value.

Time = (number of states) × (work per state).

Worked Example: Fibonacci: From $\Theta(\varphi^n)$ to $\Theta(n)$

Naive: `fib(n) = fib(n-1) + fib(n-2)` recomputes `fib(n-2)` twice, `fib(n-3)` three times, ...: exponential.

```
def fib(n, memo={}):                          # memoisation (top-down)
    if n < 2: return n
    if n not in memo: memo[n] = fib(n-1, memo) + fib(n-2, memo)
    return memo[n]

def fib_tab(n):                               # tabulation (bottom-up), O(1) space
    a, b = 0, 1
    for _ in range(n): a, b = b, a + b
    return a
```

$n+1$ states, $O(1)$ each: $\Theta(n)$.

Worked Example: 0/1 Knapsack

Same items as before: $(w, v) = (1,1), (3,4), (4,5), (5,7)$; $W = 7$. State: $\text{dp}[i][c]$ = best value using the first i items with capacity c. Recurrence: $\text{dp}[i][c] = \max(\text{dp}[i-1][c],\ \text{dp}[i-1][c-w_i] + v_i)$ if $w_i \le c$, else $\text{dp}[i-1][c]$. Base: $\text{dp}[0][c] = 0$.

$i \backslash c$	0	1	2	3	4	5	6	7
0 (none)	0	0	0	0	0	0	0	0
1 (1,1)	0	1	1	1	1	1	1	1
2 (3,4)	0	1	1	4	5	5	5	5
3 (4,5)	0	1	1	4	5	6	6	**9**
4 (5,7)	0	1	1	4	5	7	8	**9**

Sample cell: $\text{dp}[3][7] = \max(\text{dp}[2][7],\ \text{dp}[2][3] + 5) = \max(5, 4+5) = 9$. **Answer: 9.** Trace back: $\text{dp}[4][7] = \text{dp}[3][7] \Rightarrow$ item 4 not taken; $\text{dp}[3][7] \neq \text{dp}[2][7] \Rightarrow$ item 3 taken, capacity $\to 3$; $\text{dp}[2][3] \neq \text{dp}[1][3] \Rightarrow$ item 2 taken, capacity $\to 0$. Items $\{2, 3\}$, weight 7, value 9—beating greedy's 8. Time $\Theta(nW)$ (*pseudo-polynomial*: polynomial in the *value* of W, exponential in its bit-length; knapsack is still NP-complete).

Worked Example: Longest Common Subsequence (LCS)

$X =$ `ABCD`, $Y =$ `ACBD`. State $L[i][j]$ = LCS length of $X[1..i]$ and $Y[1..j]$.

$$L[i][j] = \begin{cases} L[i-1][j-1] + 1 & X_i = Y_j \\ \max(L[i-1][j],\ L[i][j-1]) & \text{otherwise} \end{cases}, \qquad L[0][\cdot] = L[\cdot][0] = 0.$$

	$\varnothing$	A	C	B	D
$\varnothing$	0	0	0	0	0
A	0	**1**	1	1	1
B	0	1	1	**2**	2
C	0	1	2	2	2
D	0	1	2	2	**3**

LCS length = 3. Trace back from $L[4][4]$: D = D (diagonal) $\to L[3][3]$: C $\neq$ B, go up $\to L[2][3]$: B = B (diagonal) $\to L[1][2]$: A $\neq$ C, go left $\to L[1][1]$: A = A. LCS = `ABD` (`ACD` is another). Time $\Theta(mn)$. This is `diff`, DNA alignment, and edit distance's sibling.

Worked Example: Coin Change (Minimum Coins) — Where Greedy Failed

Coins $\{1, 3, 4\}$, amount 6. $\text{dp}[a] = 1 + \min_{c \le a} \text{dp}[a-c]$, $\text{dp}[0] = 0$. $\text{dp}[1] = 1$, $\text{dp}[2] = 2$, $\text{dp}[3] = 1$, $\text{dp}[4] = 1$, $\text{dp}[5] = 1 + \min(\text{dp}[4], \text{dp}[2], \text{dp}[1]) = 2$, $\text{dp}[6] = 1 + \min(\text{dp}[5], \text{dp}[3], \text{dp}[2]) = 1 + 1 = \mathbf{2}$ (two 3s). Greedy said 3.

Exam Tip & Key takeaway: DP Problems to Recognise

Fibonacci, 0/1 knapsack, LCS / edit distance, coin change, matrix-chain multiplication, longest increasing subsequence, rod cutting, Floyd–Warshall (all-pairs shortest paths, $\text{d}^{(k)}[i][j] = \min(\text{d}^{(k-1)}[i][j],\ \text{d}^{(k-1)}[i][k] + \text{d}^{(k-1)}[k][j])$), Bellman–Ford, Viterbi (HMMs), CKY parsing. **DP vs divide and conquer:** both use optimal substructure; DP is needed when subproblems *overlap*. **DP vs greedy:** DP evaluates all choices at each step; greedy commits to one.

1.4 Backtracking

Template

Backtracking is depth-first search over a tree of *partial solutions*. Extend the partial solution one decision at a time; if a constraint is violated, **prune** (abandon this branch) and go back to the previous decision.

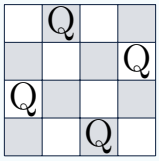
```
def backtrack(partial):
    if is_complete(partial): record(partial); return
    for choice in candidates(partial):
        if is_valid(partial, choice):          # prune early
            partial.append(choice)
            backtrack(partial)
            partial.pop()                      # undo = "back track"
```

Worst case is exponential (it explores the whole tree), but pruning often makes it practical. **Branch and bound** is the optimisation variant: also prune any branch whose *best possible* value (a bound) cannot beat the best complete solution found so far.

Worked Example: 4-Queens

Place 4 queens on a 4×4 board, one per row, no two attacking (same column or diagonal). Represent a solution as $[c_0, c_1, c_2, c_3]$ = column of the queen in each row.

Partial	Outcome
$[0]$	row 1: col 0 (same col) ×, col 1 (diag) ×, col 2 ✓
$[0, 2]$	row 2: cols 0, 1, 2, 3 all attacked ⇒ **backtrack**
$[0, 3]$	row 2: col 1 ✓
$[0, 3, 1]$	row 3: all attacked ⇒ backtrack; row 2 exhausted ⇒ backtrack; row 1 exhausted ⇒ backtrack
$[1]$	row 1: col 3 ✓ (0 diag, 1 col, 2 diag)
$[1, 3]$	row 2: col 0 ✓
$[1, 3, 0]$	row 3: col 2 ✓
$[1, 3, 0, 2]$	**solution**



Brute force would test $4^4 = 256$ placements (or $4! = 24$ permutations); backtracking with pruning examined about 8 partial boards. The other solution is the mirror image $[2, 0, 3, 1]$.

8-Queens has 92 solutions; brute force $8^8 \approx 1.7 \times 10^7$; backtracking explores about 2000 nodes.

Worked Example: Subset Sum by Backtracking

Set $\{3, 5, 7, 9\}$, target 15. Sort ascending; keep a running sum and prune when sum $>$ target or when sum + (remaining total) $<$ target. Include 3 $\rightarrow$ include 5 (8) $\rightarrow$ include 7 (15) **found** $\{3, 5, 7\}$. Continue for all solutions: 3, 5, exclude 7, include 9 ($17 > 15$) prune. 3, exclude 5, 7 (10), 9 (19) prune; 3, 7, exclude 9: $10 + 0 < 15$ prune. Exclude 3: 5, 7 (12), 9 (21) prune; 5, 9 (14) $\neq 15$; 7, 9 (16) prune. Only $\{3, 5, 7\}$.

Where Computer Science Uses This: Backtracking Everywhere

Sudoku and crossword solvers, graph colouring (Unit IV), Hamiltonian cycle search, DPLL SAT solving (Unit V), regular-expression engines, constraint programming, game-tree search (minimax with alpha–beta pruning is backtracking with bounds), and parsing.

1.5 Genetic Algorithms

Evolution as an Optimiser

A **genetic algorithm (GA)** maintains a *population* of candidate solutions (*chromosomes*), scores them with a *fitness function*, and produces the next generation by *selection* (fitter individuals reproduce more), *crossover* (children mix parents' genes) and *mutation* (random small changes). Over generations the population drifts toward high fitness. No gradient, no convexity, no continuity required—only the ability to evaluate fitness.

Formula & Mathematical Relation: GA Components

1. **Representation (encoding):** binary string, integer vector, permutation (for TSP), real vector, tree (genetic programming).
2. **Initial population:** N random chromosomes (typically 50–500).
3. **Fitness function** $F(\cdot)$: the objective (maximise).
4. **Selection:** *roulette wheel* (probability $p_i = F_i / \sum_j F_j$), *tournament* (pick k at random, keep the best), *rank-based*. *Elitism*: copy the best few unchanged.
5. **Crossover** (rate $p_c \approx 0.6$–0.9): single-point, two-point, uniform; order crossover (OX) or PMX for permutations.
6. **Mutation** (rate $p_m \approx 0.001$–0.05 per gene): flip a bit, perturb a real, swap two positions.
7. **Termination:** fixed number of generations, fitness threshold, or no improvement for g generations.

Worked Example: One Generation of a GA: Maximise $f(x) = x^2$ on $x \in [0, 31]$

Encode x as a 5-bit string. Initial population and fitness:

#	Chromosome	x	$f = x^2$	$p_i = f_i / \sum f$	Expected copies
1	01101	13	169	0.14	0.58
2	11000	24	576	0.49	1.97
3	01000	8	64	0.06	0.22
4	10011	19	361	0.31	1.23
$\sum f = 1170$, average $= 293$, max $= 576$					

Expected copies $= p_i N$ with population size $N = 4$. **Selection (roulette):** rounding the expected counts, the mating pool is $\{01101, 11000, 11000, 10011\}$ (chromosome 3 dies out). **Crossover (single point):** Pair $(01101, 11000)$ at point 4: $0110|1$ and $1100|0 \to 01100$ (12) and 11001 (25). Pair $(11000, 10011)$ at point 2: $11|000$ and $10|011 \to 11011$ (27) and 10000 (16). **Mutation:** with $p_m = 0.001$ per bit and 20 bits, expect 0.02 flips—assume none this generation. **New population:** $01100, 11001, 11011, 10000$ with fitness $144, 625, 729, 256$: $\sum = 1754$, average $293 \to 439$, best $576 \to 729$. One generation has improved both the average and the best; the optimum 11111 ($x = 31$, $f = 961$) is now reachable by a single mutation of 11011.

Common Pitfall & Misconception: GA Failure Modes

- **Premature convergence:** one super-fit individual takes over the population; diversity collapses. Fixes: tournament or rank selection, higher mutation, larger population, niching.
- **Bad encoding:** crossover of two valid TSP tours as plain lists produces invalid tours (repeated cities); use permutation-preserving operators.
- **Fitness scaling:** with roulette selection, adding a constant to all fitnesses changes the selection pressure.
- **No guarantee:** a GA can return a poor solution and will not tell you. Run multiple seeds; compare with a bound.

1.6 Nature-Inspired Algorithms and Approaches

The Common Pattern: Exploration vs. Exploitation

All metaheuristics balance **exploration** (sampling new regions of the search space so as not to get stuck in a local optimum) against **exploitation** (refining the best solutions found so far). Nature supplies many metaphors for this balance. The **No Free Lunch theorem** (Wolpert & Macready, 1997) says that averaged over *all* possible problems, every optimiser performs equally—so the choice must be matched to the structure of the problem at hand.

Formula & Mathematical Relation: Simulated Annealing (SA)

Inspired by cooling metal. Single solution s; at temperature T, propose a neighbour s' with cost change $\Delta = C(s') - C(s)$:

$$P(\text{accept}) = \begin{cases} 1 & \Delta \le 0 \\ e^{-\Delta/T} & \Delta > 0 \end{cases} \tag{1}$$

Then cool: $T \leftarrow \alpha T$ ($\alpha \approx 0.9$–0.99). High T: accepts almost anything (exploration); low T: pure hill-climbing (exploitation). Uphill moves let it escape local minima.
Example: $\Delta = +3$. At $T = 10$: $P = e^{-0.3} = 0.74$. At $T = 1$: $P = e^{-3} = 0.05$. At $T = 0.1$: $P \approx 10^{-13}$.

Formula & Mathematical Relation: Particle Swarm Optimisation (PSO)

Inspired by bird flocking. Each particle i has position $\mathbf{x}_i$ and velocity $\mathbf{v}_i$, remembers its personal best $\mathbf{p}_i$, and knows the swarm's global best $\mathbf{g}$:

$$\mathbf{v}_i \leftarrow w\,\mathbf{v}_i + c_1 r_1(\mathbf{p}_i - \mathbf{x}_i) + c_2 r_2(\mathbf{g} - \mathbf{x}_i) \tag{2}$$

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i \tag{3}$$

$w =$ inertia (≈ 0.7), $c_1, c_2 \approx 2$ (cognitive and social weights), $r_1, r_2 \sim U(0, 1)$. Continuous optimisation; few parameters; used for neural-network weight tuning and antenna design.

One step in 1-D: $x = 5$, $v = 1$, $p = 6$, $g = 9$, $w = 0.7$, $c_1 = c_2 = 2$, $r_1 = 0.5$, $r_2 = 0.25$: $v = 0.7 + 2(0.5)(1) + 2(0.25)(4) = 0.7 + 1 + 2 = 3.7$; $x = 8.7$.

Formula & Mathematical Relation: Ant Colony Optimisation (ACO)

Inspired by ants laying pheromone on short paths. For combinatorial problems on graphs (TSP, routing). An ant at i chooses next city j with probability

$$p_{ij} = \frac{\tau_{ij}^{\alpha}\,\eta_{ij}^{\beta}}{\sum_{k \in \text{allowed}} \tau_{ik}^{\alpha}\,\eta_{ik}^{\beta}}, \qquad \eta_{ij} = 1/d_{ij} \text{ (heuristic desirability).} \tag{4}$$

After all ants finish a tour, pheromone evaporates and is reinforced by good tours:

$$\tau_{ij} \leftarrow (1-\rho)\,\tau_{ij} + \sum_{\text{ants } k} \Delta\tau_{ij}^{k}, \qquad \Delta\tau_{ij}^{k} = Q/L_k \text{ if ant } k \text{ used edge } (i,j). \tag{5}$$

Evaporation (ρ) is the exploration term; reinforcement is exploitation. Used for vehicle routing and network packet routing (AntNet).

Algorithm	Inspiration	Key mechanism	Best suited to
Genetic algorithm	Darwinian evolution	Selection, crossover, mutation on a population	Discrete/mixed, permutations, any black box
Simulated annealing	Metallurgy	Single solution; accept worse moves with $e^{-\Delta/T}$	Combinatorial (VLSI placement, scheduling)
Particle swarm	Bird flocks	Velocity toward personal and global best	Continuous parameter tuning
Ant colony	Ant foraging	Pheromone trails on graph edges	Path / routing / TSP
Artificial bee colony	Honey-bee foraging	Employed, onlooker and scout bees explore food sources	Continuous, numerical
Firefly	Bioluminescence	Attraction proportional to brightness, decays with distance	Multimodal continuous
Cuckoo search	Brood parasitism	Lévy-flight random walks; worst nests abandoned	Global continuous optimisation
Grey wolf optimiser	Pack hunting hierarchy	Alpha/beta/delta lead the encircling of prey	Engineering design
Differential evolution	—	Mutation by scaled difference of random vectors	Continuous; robust default

Table 2: Nature-inspired metaheuristics. Population-based methods (GA, PSO, ACO, ABC) parallelise well; trajectory methods (SA, tabu search) are cheap per iteration.

Exam Tip & Key takeaway: Exact vs. Heuristic: Making the Call

- Problem in P with a known algorithm (sorting, shortest path, MST, matching) → use it; never use a GA to sort.
- NP-hard but small ($n \lesssim 30$) or highly constrained → backtracking / branch and bound, or a SAT/ILP solver.
- NP-hard, large, objective cheap to evaluate → metaheuristic (GA for discrete, PSO/DE for continuous, ACO for routing, SA when a good neighbourhood exists).
- Report the *gap* to a lower bound (LP relaxation, MST bound for TSP) so the "good enough"

claim is quantified.

Where Computer Science Uses This: Optimisation in Industry

Airline crew scheduling (branch and bound + heuristics), chip floor-planning (simulated annealing), hyperparameter search for ML models (GA / Bayesian optimisation), antenna and wing design (evolutionary algorithms produce shapes no human would draw), logistics routing (ACO and local search), portfolio optimisation (PSO), protein folding and drug docking (GA), and neural architecture search (evolutionary methods).

1.7 Unit VI Quick-Revision Sheet

Exam Tip & Key takeaway: Ten Things to Carry Into the Exam

1. Greedy needs the greedy-choice property + optimal substructure; prove by exchange argument. Works: activity selection (earliest finish), fractional knapsack (ratio), Huffman, MST, Dijkstra. Fails: 0/1 knapsack, coin change $\{1, 3, 4\}$.
2. Divide and conquer: independent subproblems; $T(n) = aT(n/b) + f(n)$ via master theorem. Karatsuba 3 products $\to n^{1.585}$; Strassen 7 $\to n^{2.81}$.
3. DP: overlapping subproblems + optimal substructure. Define state, recurrence, base, order, traceback. Time = states $\times$ work per state.
4. Knapsack $\text{dp}[i][c] = \max(\text{dp}[i-1][c], \text{dp}[i-1][c-w_i] + v_i)$, $\Theta(nW)$ pseudo-polynomial.
5. LCS: diagonal $+1$ on match, else max(up, left), $\Theta(mn)$.
6. Backtracking = DFS on partial solutions + pruning; branch and bound adds a bound for optimisation. N-Queens, subset sum, SAT, Sudoku.
7. GA loop: evaluate $\to$ select (roulette $p_i = f_i / \sum f$, tournament) $\to$ crossover $\to$ mutate $\to$ replace (elitism). Encoding must suit the operators.
8. SA accepts worse moves with $e^{-\Delta/T}$; cool geometrically. PSO: $v \leftarrow wv + c_1 r_1 (p - x) + c_2 r_2 (g - x)$. ACO: $p_{ij} \propto \tau^{\alpha} \eta^{\beta}$, evaporate then reinforce.
9. Exploration vs exploitation is the single knob in every metaheuristic; No Free Lunch: no universally best optimiser.
10. Use exact methods when they exist; metaheuristics for large NP-hard instances; always report the gap to a bound.